Тема 5. Типы данных линейной структуры

Лекция 10

Целью данной **темы** является изучение типов данных линейной структуры. В ней рассматриваются такие линейные структуры данных как списки, стеки, очереди и хеш-таблицы. Рассматриваются вопросы их программной реализации.

5.1. Связные линейные списки

Списком называется упорядоченное множество, состоящее из переменного числа элементов, к которым применимы операции включения, исключения. Список, отражающий отношения соседства между элементами, называется линейным. *Длина* списка равна числу элементов, содержащихся в списке, список нулевой длины называется *пустым списком*. Линейные связные списки являются простейшими динамическими структурами данных.

В целом, структура списка похожа на модель звеньев в цепи. Длина списка может увеличиваться при добавлении новых звеньев. Новые элементы могут быть вставлены в список простым разрывом соединения, добавлением нового звена и восстановлением соединения. Элементы удаляются из списка путем разрыва двух соединений, удаления звена и затем повторного соединения цепи.

Списки бывают трех видов:

- односвязные;
- циклические;
- двусвязные.

5.1.1. Односвязный линейный список

Односвязный линейный список может быть реализован двумя способами – на основе массивов, или при помощи указателей.

При реализации списков на основе массивов элементы списка располагаются в смежных ячейках массива. Это представление позволяет легко просматривать содержимое списка и вставлять новые элементы в его конец. Но чтобы вставить новый элемент в середину списка, необходимо освободить для него место путем перемещения всех последующих элементов на одну позицию к концу массива. Удаление элемента также требует перемещения элементов, чтобы закрыть ячейку, которая освободилась.

Но данный подход имеет несколько недостатков:

- реализация списков на основе массивов требует указания максимального размера списка до начала выполнения программ;
- реализация списков на основе массивов требует большего объема компьютерной памяти. Чтобы реализовать список, необходимо выделить объем памяти, достаточный для максимально возможного размера списка.

На рис. 5.1 приведена схема реализации списка на массиве.

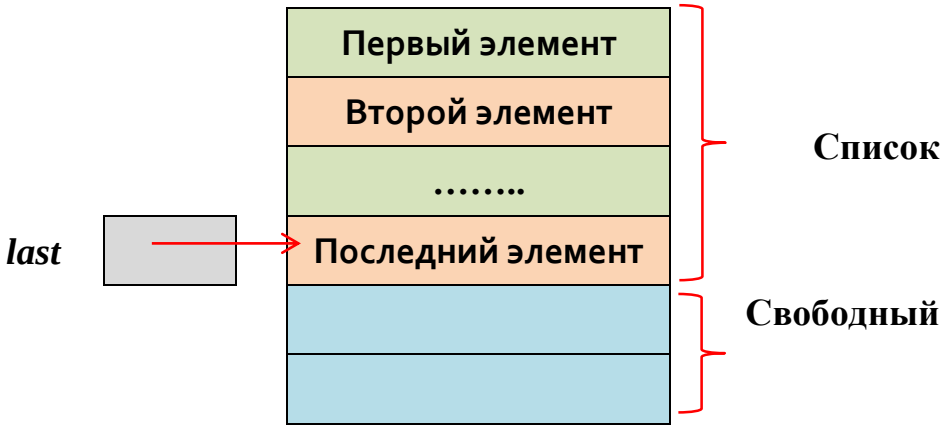


Рис. 5.1. Схема реализации списка на массиве

Реализация списков при помощи указателей освобождает нас от использования непрерывной области памяти для хранения списка. Поэтому при данном подходе нет необходимости перемещать элементы списка при вставке или удалении элементов. Однако при этом требуется дополнительная память для хранения указателей.

Каждый узел списка состоит из поля данных и указателя, обозначающего следующий элемент в списке. Поле указателя последнего элемента списка имеет значение *NULL*.

Структура связанного списка может быть представлена следующим образом:

```
struct LIST{
        int dann;
        LIST *next;
};
```

Для описания списка необходимо три указателя:

1. *head* – указатель на первый элемент (на голову) списка;
2. *rear* – указатель на последний элемент (на хвост) списка;
3. *ptr* – указатель на текущий элемент списка.

При начальной инициализации все три указателя принимают значение NULL:

```
LIST *head=NULL;
LIST *rear=NULL;
LIST *ptr=NULL;
```

Для списка без элементов *head* будет содержать значение *NULL*. Для доступа к элементу списка необходим просмотр списка с головы.

На рис. 5.2 приведена схема реализации списка при помощи указателей.

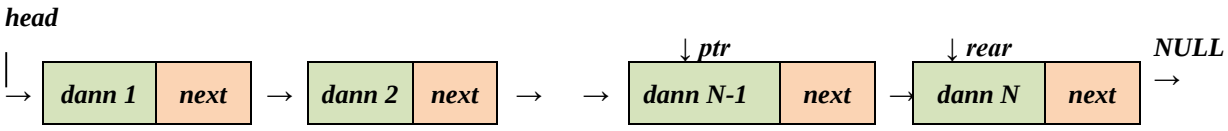


Рис. 5.2. Схема реализации односвязного списка при помощи указателей

Для списка должны быть определены следующие операции:

- формирование списка;
- прохождение по связному списку;
- очистка списка;
- вставка элемента после определенного узла;
- удаление элемента, следующего после определенного узла.

Рассмотрим их реализацию.

Операция формирования списка. В данном примере добавление элементов списка производится в хвосте.

```
void make(int  a) {
        LIST *ptr;
        // выделяется память для нового элемента списка
        ptr=new LIST;
        // если это будет первый элемент в списке, то на
        // него должен указывать указатель head
        if (!head) head=ptr;
        // иначе, присоединяем элемент к существующему списку
        else rear->next=ptr;
        // заполняем поле данных нового элемента
        ptr->dann=a;
        // т.к. элемент добавляется в конец списка, то на
        // этот элемент должен указывать указатель rear
        rear=ptr;
        // поле указателя элемента rear
        // должно содержать значение NULL
        rear->next=NULL;
}
```

Операция прохождения по связному списку:

```
void print(void)  {
        // указатель ptr устанавливается на начало списка
        LIST *ptr=head;
        // указатель ptr перемещается к последующему узлу,
        // пока не достигнем конца списка
        while (ptr){
        // в процессе прохождения по списку
        // печатаются элементы списка
                cout << ptr->dann << " ";
                ptr=ptr->next;
        }
        cout << endl;
}
```

Операция очистки списка:

```
void clear(void) {
        LIST *ptr;

        // указатель head перемещается к последующему
        // узлу, пока не достигнем конца списка
        while (head) {
                ptr=head->next;
                // в процессе прохождения по списку удаляется
                // элемент, который находится в начале списка
                delete head;
                head=ptr;
        }
}
```

Операция вставки элемента после определенного узла. Схема вставки элемента приведена на рис. 5.3.

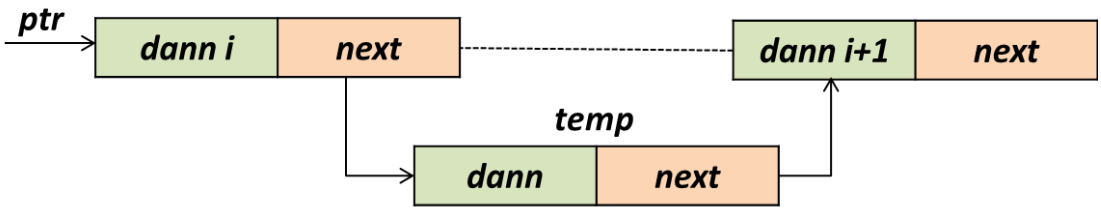


Рис. 5.3. Схема вставки элемента в связный список

Следующий программный код реализует схему вставки элемента:

```
// num - номер элемента, после которого производится вставка.
// a - значение поля данных.
void insert(int num, int a) {
        LIST *ptr=head;
        LIST *temp;
        int i=0;
        // указатель ptr перемещается по списку до тех пор, пока номер
        // элемента не станет равным введенному номеру
        while (i++!=num)   ptr=ptr->next;
        // затем производится вставка узла согласно схеме.
        temp=new LIST;
        temp->dann=a;
        temp->next=ptr->next;
        ptr->next=temp;
}
```

Операция удаления элемента, следующего после определенного узла. Схема удаления элемента приведена на рис. 5.4.

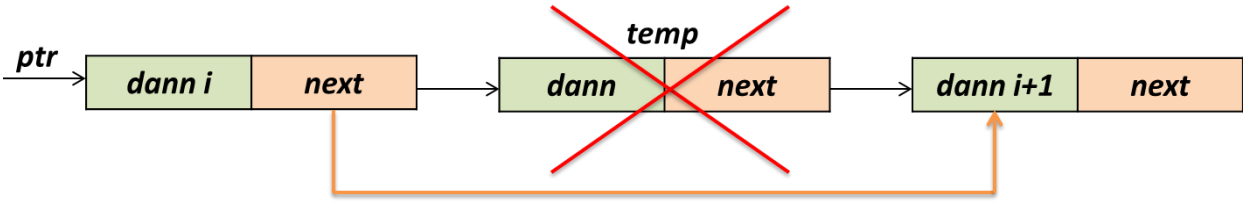


Рис. 5.4. Схема удаления элемента из связного списка

Следующий программный код реализует схему удаления элемента:

```
void deleten(int num) {
        LIST *ptr=head;
        LIST *temp;
        int i=0;
        // Указатель ptr перемещается по списку до тех пор,
        // пока номер элемента не станет равным
        // введенному номеру num.
         while (i++!=num)       ptr=ptr->next;
        // Указатель temp указывает на удаляемый элемент списка.
        temp=ptr->next;
        // Удаление элемента производится согласно схеме
        ptr->next=temp->next;
        delete temp;
}
```

Во многих приложениях необходимо использовать упорядоченный список данных. Его элементы должны располагаться в возрастающем или убывающем порядке. Чтобы определить правильное положение нового элемента, алгоритм вставки должен сканировать список. Затем производится вставка.

Рассмотрим следующий пример. Допустим, список первоначально содержит целые числа 60, 65, 74, 82 (рис. 5.5). Необходимо вставить в список числа 50, 70, 90.

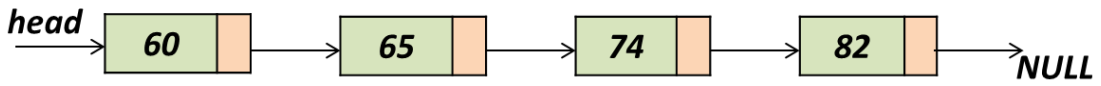


Рис. 5.5. Исходный список

Вставка числа 50 в список. Первый элемент в списке – это 60. Его значение больше чем 50. Поэтому новый элемент вставляем в начало списка. Этот процесс представлен на рис. 5.6.

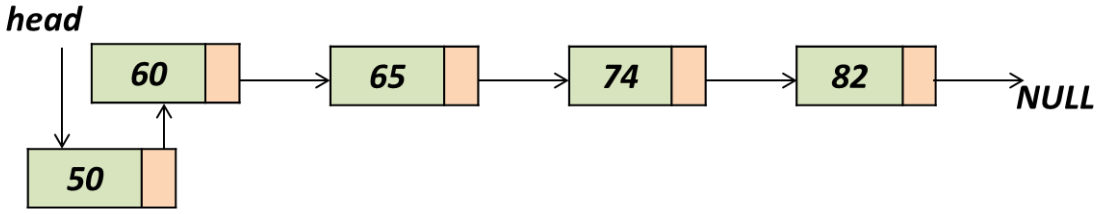


Рис. 5.6. Вставка числа 50 в список

Вставка числа 70 в список. 74 – это первый элемент в списке, больший, чем 70. Указатели *pr_ptr* и *cur_ptr* обозначают элементы 65 и 74 соответственно. Вставка производится согласно схеме, представленной на рис. 5.7.

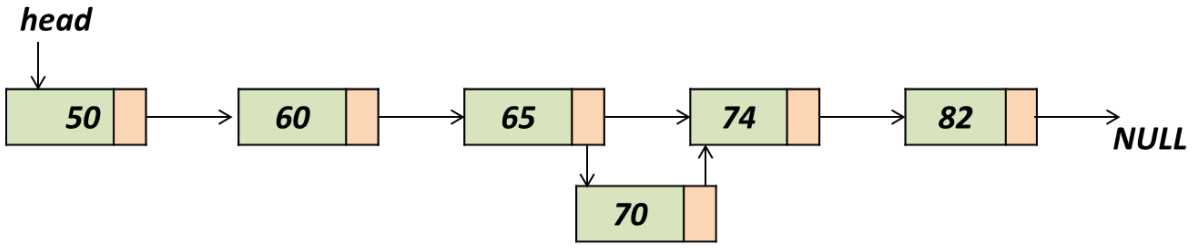


Рис. 5.7. Вставка числа 70 в список

Вставка числа 90 в список. Проверяется весь список. В нем нет элемента со значением больше чем 90 (*cur_ptr==NULL*). Значит, новое

значение должно быть помещено в конец списка после *pr_ptr*. Иллюстрация этого представлена на рис. 5.8.

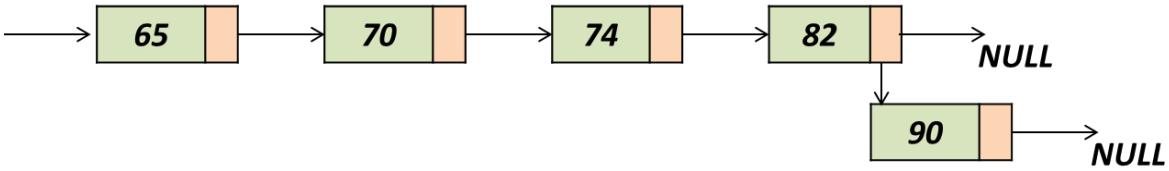


Рис. 5.8. Вставка числа 90 в список

5.1.2. Циклические списки

До сих пор мы рассматривали линейные списки, в которых последний элемент списка содержал значение указателя *NULL*. Если заменить значение указателя последнего элемента на адрес начала списка, то получим циклический список. Его структура представлена на рис. 5.9.

Циклический список является более гибкой структурой. Он позволяет начинать обход списка в любом положении и продолжать его до начальной позиции.

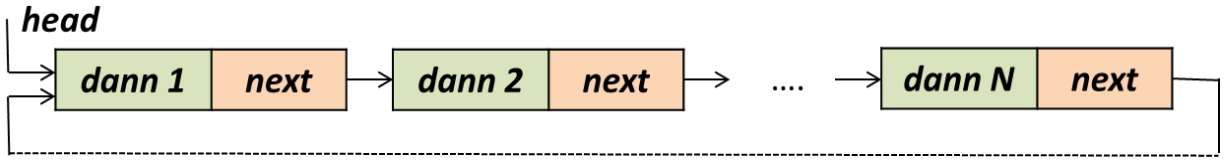


Рис. 5.9. Структура циклического списка

Многие профессиональные программисты используют циклическую модель для реализации связанных списков. Достоинство циклического списка – это доступность элементов (можно из любого узла списка попасть в любой другой узел посредством передвижения по списку). Недостаток заключается в том, что при неосторожном прохождении по списку можно попасть в бесконечный цикл.

5.1.3. Двусвязный линейный список

В некоторых задачах необходимо иметь возможность двигаться по списку в двух направлениях. Это можно сделать, если ввести в каждый элемент списка два поля связи вместо одного. В этих полях находятся адреса предыдущего и последующего элементов.

Линейный список называется списком с двумя связями, или двусвязным списком, если каждый элемент этого списка имеет два указателя (ссылки на предыдущий и последующий элементы списка).

В программе двусвязный список можно реализовать при помощи следующей структуры:

```
struct NDD  {
        int val;                    /* значение элемента  */
        struct NDD * next;          /*указатель на следующий элемент */
        struct NDD * prev;          /*указатель на предыдущий элемент*/
   };
NDD * beg=NULL, * end=NULL, * rex=NULL;
```

Графически двусвязный список можно представить так, как показано на рис. 5.10.

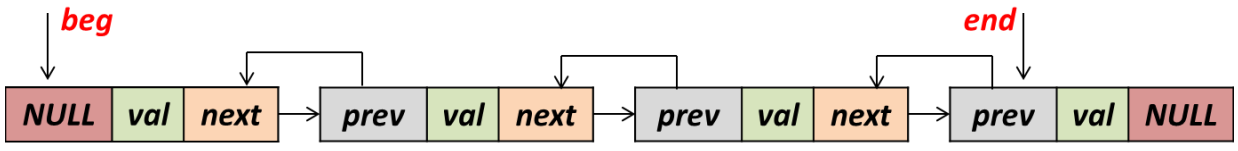


Рис. 5.10. Структура двусвязного списка

Формирование двусвязного списка можно реализовать следующим образом:

```
// выделяется память для нового элемента списка
rex=new NDD;
// заполняем поле данных нового элемента
printf("Введите данные  ");
scanf("%f",&rex->val);
// если это будет первый элемент в списке,
// то на него должен указывать указатель beg
if (beg==NULL)   beg=rex;
// иначе, присоединяем элемент к существующему списку
else end->next=rex;
rex->prev=end;
// т.к. элемент добавляется в конец списка, то на этот элемент
// должен указывать указатель end
end=rex;
// поле указателя элемента end должно содержать значение NULL
end->next=NULL;
```

Удаление элемента из двусвязного списка можно представить так:

```
float ad;
printf("Введите значение элемента, который надо удалить: ");
scanf("%f",&ad);  i=1;
rex=beg;
while(rex!=NULL&&rex->val!=ad) rex=rex->next;
if (rex->next==NULL) printf("Такого элемента нет");
else {rex->prev->next=rex->next;
rex->next->prev=rex->prev;
delete rex;
```

На рис. 5.11 приведена схема удаления элемента из двусвязного списка.

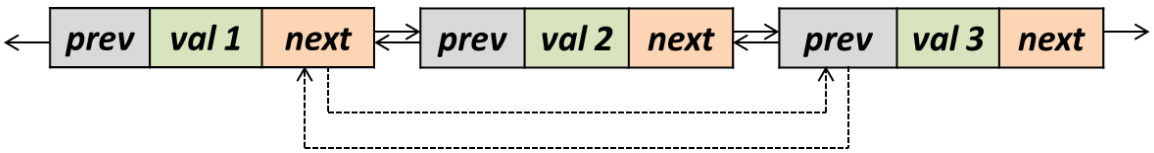


Рис. 5.11. Схема удаления элемента из двусвязного списка

Оператор

rex->prev->next=rex->next;

можно представить как

(rex->prev)->next=rex->next;

Такой оператор означает, что в поле *next* элемента, стоящего перед удаляемым элементом, заносится адрес элемента, который находится после удаляемого элемента.

Возможно и построение двусвязного кольцевого списка. Для этого пустые указатели двусвязного линейного списка необходимо заменить указателями противоположных концов списка. При этом нет необходимости в указателе конца списка *end*. На рис. 5.12 приведена структура двусвязного кольцевого списка.

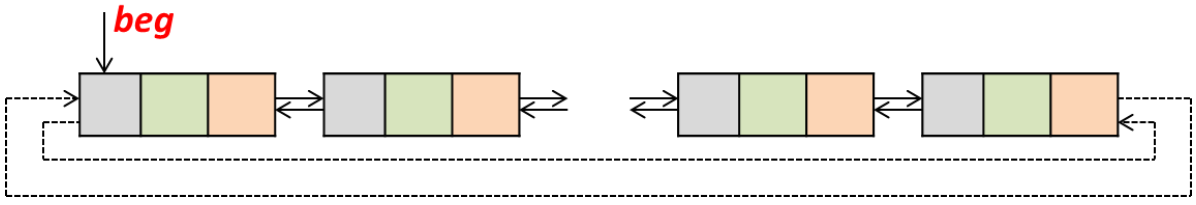


Рис. 5.12. Структура двусвязного кольцевого списка

Массив	Список
Выделение памяти осуществляется единовременно под весь массив до начала его использования	Выделение памяти осуществляется по мере ввода новых элементов
При удалении/добавлении элемента требуется копирование всех последующих элементов для осуществления их сдвига	Удаление/добавление элемента осуществляется переустановкой указателей, при этом сами данные не копируются
Для хранения элемента требуется объем памяти, необходимый только для хранения данных этого элемента	Для хранения элемента требуется объем памяти, достаточный для хранения данных этого элемента и указателей (1 или 2) на другие элементы списка
Доступ к элементам может осуществляться в произвольном порядке	Возможен только последовательный доступ к элементам

Рис. 5.13. Сравнение массивов и связных списков